Data Structures (CS2103)

Stack Applications

Prepared By – Ashish K Layek

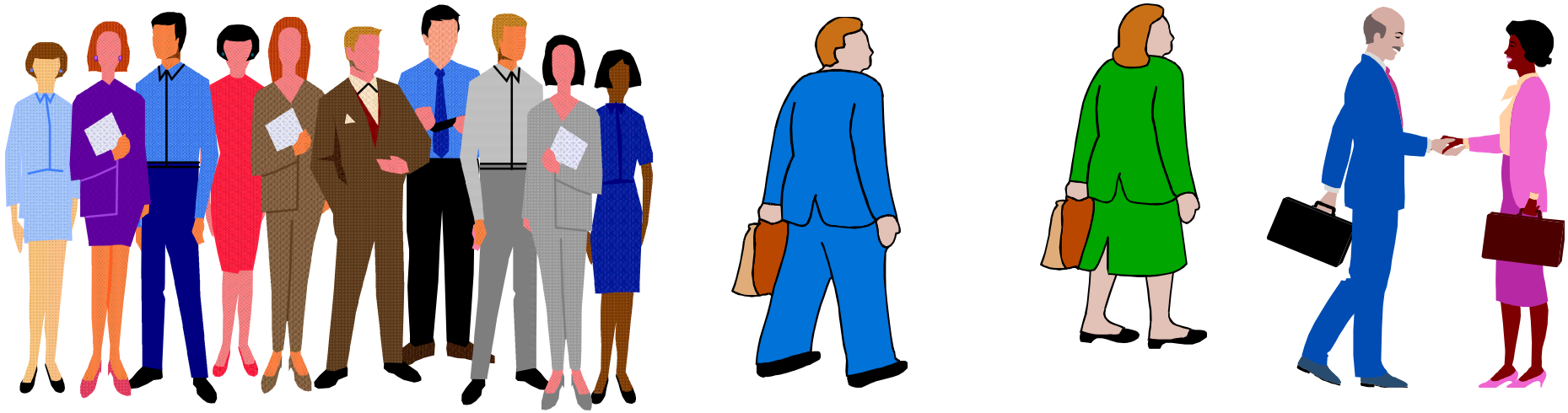
CST, IIEST Shibpur

Recursion

Example 1: The Handshake Problem

There are n people in a room. If each person shakes hands once with every other person. What is the total number **h(n)** of handshakes?

h(n) = h(n-1) + n-1 **h(4) = h(3) + 3** **h(3) = h(2) + 2** **h(2) = 1**



h(n): Sum of integer from 1 to n-1 = n(n-1) / 2

Recursion

- In some problems, it may be natural to define the problem in terms of the problem itself.

- Recursion is useful for problems that can be represented by a simpler version of the same problem.

  - Example: the factorial function

    ```
    6! = 6 * 5 * 4 * 3 * 2 * 1
    ```

    We could write:

    ```
    6! = 6 * 5!
    ```

Example 2: Factorial problem

- In general, we can express the factorial function as follows:

```
n! = n * (n-1)!
```

- Is this correct? Well… almost.

- The factorial function is only defined for *positive* integers. So we should be a bit more precise:

```
n! = 1     (if n is equal to 1)
n! = n * (n-1)!   (if n is larger than 1)
```

Factorial function

The C equivalent of this definition:

```
int fac(int numb){
    if(numb <= 1)
        return 1;
    else
        return numb * fac(numb-1);
}
```

- ***recursion*** **means that a function calls itself**

Factorial function

Assume the number typed is 3, that is, numb=3.

```
fac(3) :
  3 <= 1 ?          No.
  fac(3) = 3 * fac(2)

    fac(2) :
       2 <= 1 ?       No.
       fac(2) = 2 * fac(1)

          fac(1) :
             1 <= 1 ?     Yes.
             return 1

       fac(2) = 2 * 1 = 2
       return fac(2)

  fac(3) = 3 * 2 = 6
  return fac(3)

  fac(3)  has the value 6
```

```
int fac(int numb){
  if(numb <= 1)
      return 1;
  else
      return numb * fac(numb-1);
}
```

Factorial function

For certain problems (such as the factorial function), a recursive solution often leads to short and elegant code. Compare the recursive solution with the iterative solution:

Recursive solution

```
int fac(int numb){
   if(numb<=1)
     return 1;
   else
     return numb*fac(numb-1);
}
```

Iterative solution

```
int fac(int numb){
   int product=1;
   while(numb>1) {
       product *= numb;
       numb--;
   }
   return product;
}
```

Recursion

- We have to pay a price for recursion:
  - calling a function consumes more time and memory than adjusting a loop counter.
  - high performance applications (graphic action games, simulations of air traffic, rocket, nuclear explosions) hardly ever use recursion.

- In less demanding applications recursion is an attractive alternative for iteration (for the right problems!)

Recursion

- We must always make sure that the recursion *bottoms out*:
  - A recursive function must contain at least one non-recursive branch.
  - The recursive calls must eventually lead to a non-recursive branch.

- Recursion is one way to decompose a task into smaller subtasks. At least one of the subtasks is a smaller example of the same task.
  - The smallest example of the same task has a non-recursive solution.

Problem: How many pairs of rabbits can be produced from a single pair in a year's time?

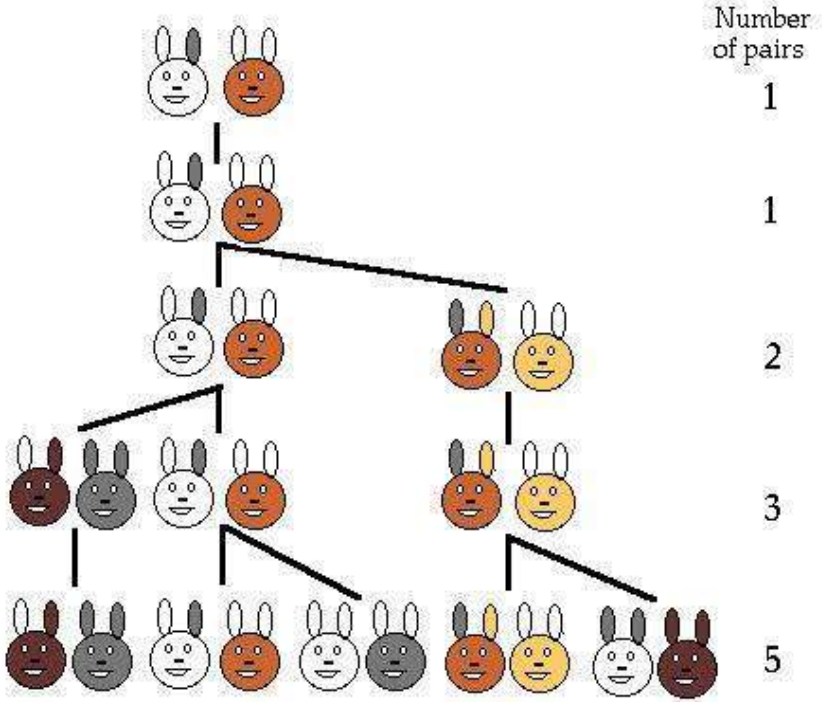
Assumptions:

- Each pair of rabbits produces a new pair of offspring every month;
- each new pair becomes fertile at the age of one month;
- none of the rabbits dies in that year.

Example:

- After 1 month there will be 2 pairs of rabbits;
- after 2 months, there will be 3 pairs;
- after 3 months, there will be 5 pairs (since the following month the original pair and the pair born during the first month will both produce a new pair and there will be 5 in all).

Population Growth in Nature



- Leonardo Pisano (Leonardo Fibonacci = Leonardo, son of Bonaccio) proposed the sequence in 1202 in *The Book of the Abacus*.
- Fibonacci numbers are believed to model nature to a certain extent, such as Kepler's observation of leaves and flowers in 1611.

Direct Computation Method

- Fibonacci Numbers:

  `0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...`

  where each number is the sum of the preceding two.

- Recursive definition:
  - `F(0) = 0;`
  - `F(1) = 1;`
  - `F(number) = F(number-1)+ F(number-2);`

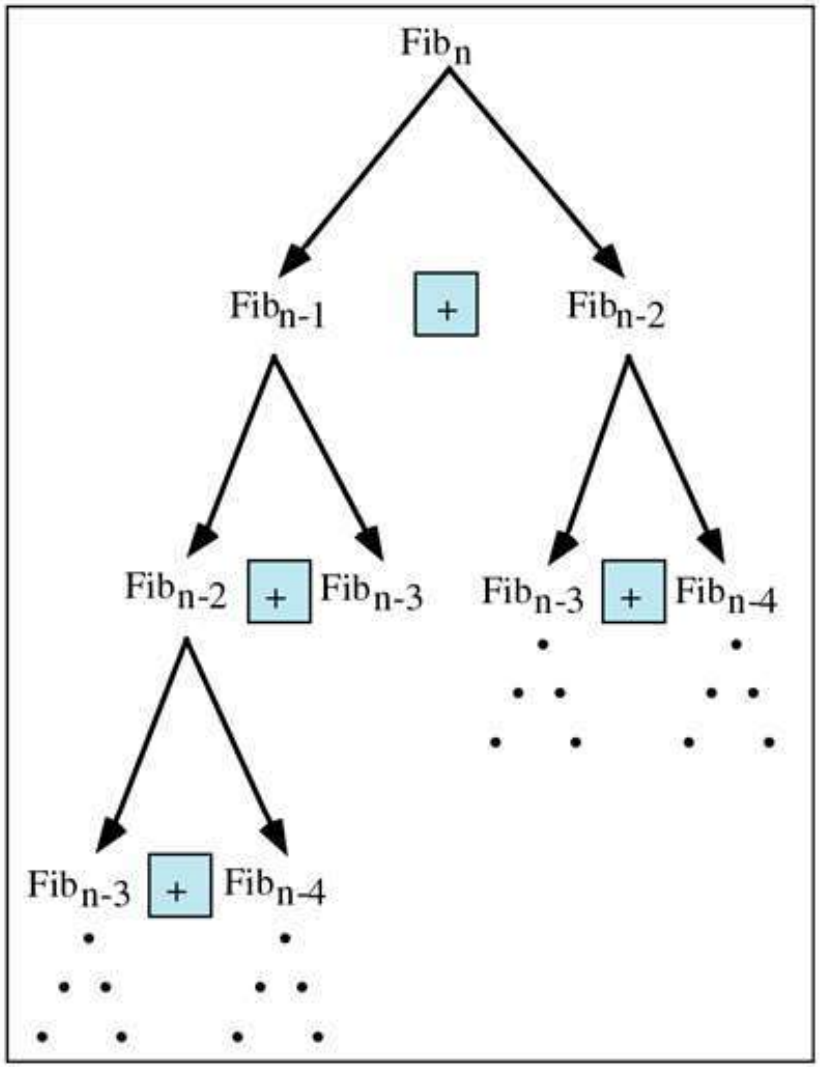
Fibonacci numbers

Calculate Fibonacci numbers using recursive function.

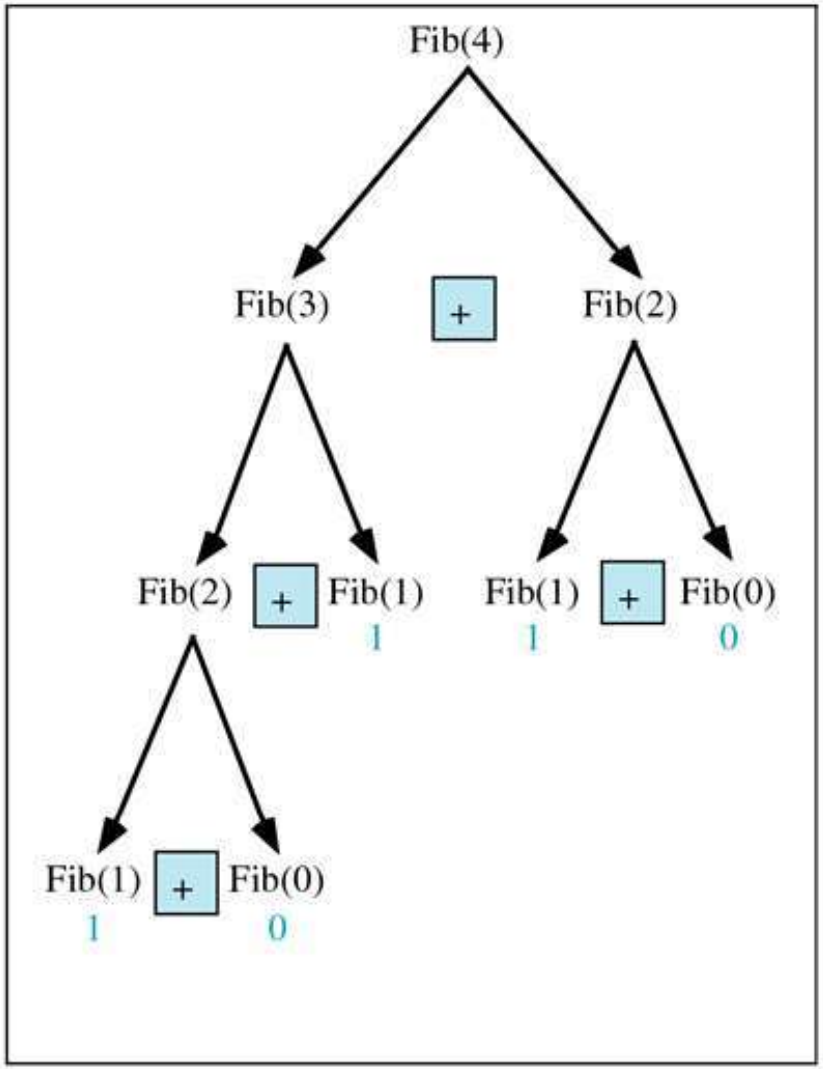
A very inefficient way, but illustrates recursion well

```
int fib(int number){
  if (number == 0) return 0;
  if (number == 1) return 1;
  return (fib(number-1) + fib(number-2));
}
```

The function call tracing



(a) Fib(n)



(b) Fib(4)

Fibonacci number w/o recursion

Calculate Fibonacci numbers iteratively

Much more efficient than recursive solution

```
int fib(int n) {
  int f[100];
  f[0] = 0; f[1] = 1;
  for (int i=2; i<= n; i++)
      f[i] = f[i-1] + f[i-2];
  return f[n];
}
```

Binary Search

- Search for an element in an array
  - Sequential search
  - Binary search
    - ✓ Elements in the array stays in sorted order

- Binary search
  - Compare the search element with the middle element of the array
  - If not equal, then apply binary search to half of the array (if not empty) where the search element would be.

Binary Search with Recursion

```
// Searches an ordered array of integers using recursion
int bsearchr(int data[], // input: array
             int first,         // input: lower bound
             int last,         // input: upper bound
             int value          // input: value to find
         ) // output: index if found,  otherwise return -1
{
   int middle = (first + last) / 2;
   if (data[middle] == value)
      return middle;
   else if (first >= last)
      return -1;
   else if (value < data[middle])
      return bsearchr(data, first, middle-1, value);
   else
      return bsearchr(data, middle+1, last, value);
}
```

Binary Search

```
#define ARRAY_SIZE = 8;
int main() {
    int list[array_size]={1, 2, 3, 5, 7, 10, 14, 17};
    int search_key = 0;
    printf("Enter search value: ");
    scanf("%d", &search_key)
    int key_pos =
            bsearchr(list,0, ARRAY_SIZE -1, search_value);
    printf("Key position : %d\n", key_pos);

    return 0;
}
```

Binary Search w/o recursion

```
// Searches an ordered array of integers
int bsearch(int data[], // input: array
            int size,    // input: array size
            int value    // input: value to find
           )             // output: if found,return
                         // index; otherwise, return -1
{
   int first, last, upper;
   first = 0;
   last = size - 1;

   while (true) {
          middle = (first + last) / 2;
          if (data[middle] == value)
               return middle;
          else if (first >= last)
               return -1;
          else if (value < data[middle])
        last = middle - 1;
          else
        first = middle + 1;
   }
}
```

Binary Search time complexity

search space size	iteration number
n	1
n/2	2
n/4	3
…	…
1	x

x = number of iterations

Hence $n/2^x = 1$

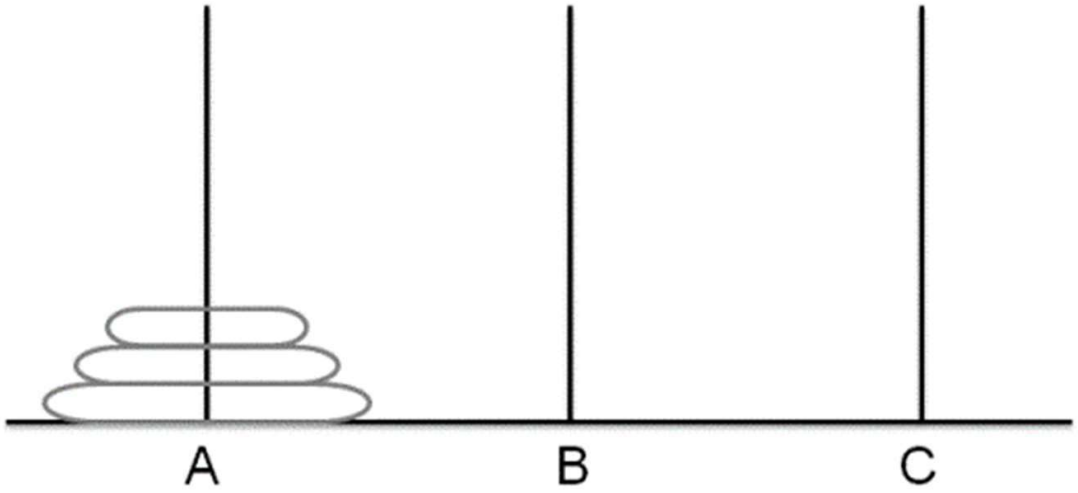
Observe $2^x = n$, $\log 2^x = \log n$,
x = log n iterations carried out

Binary Search worst-case time complexity: O(log n)

Binary Search time complexity

- Worst-case: O(log n)
  - If 'value' does not exist
- Best-case: O(1)
  - When the target element 'value' happens to be in the middle of the array
- Average-case: O(log n)
  - Technically, some statistical analysis needed here (beyond the scope of this course)

Towers of Hanoi



- Only one disc could be moved at a time
- A larger disc must never be stacked above a smaller one
- One and only one extra needle could be used for intermediate storage of discs

Towers of Hanoi

```
void towersOfHanoi(int n, char F, char T, char A) {
    // F = from, T = to, A = aux
    if (n == 1){
        printf("move disc no. 1 from %d to %d\n", F, T);
    }
    else {
       // move top n-1 disks from F to A using T
       hanoi(n-1, F, A, T);

       printf("move disc no. %d from %d to %d\n", n, F, T);

       // move top n-1 disks from A to T using F
       hanoi(n-1, A, T, F);
    }
}
```

Recursion General Form

How to write recursively?

```
int recur_fn(parameters){
  if(stopping condition)
      return stopping value;
  // other stopping conditions if needed
  return function of recur_fn(revised parameters)

}
```

In Summary

- Recursion is a powerful way to think about problems and understand what's really going on

- Often not efficient in implementation. Iterative implementation is better

Evaluation of Arithmetic Expression

Evaluation of Arithmetic Expression

An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression.

- **Infix Notation**: The operators are placed in-between operands in this notation. For example, `a + b * c.`
  - Easy for us humans to read, write, and speak but not for computers.
- **Prefix (Polish) Notation**: The operator is prefixed to operands, i.e. operator is written ahead of operands. For example, `+ a * b c.`
- **Postfix (Reverse-Polish) Notation**: The operator is postfixed to the operands i.e., the operator is written after the operands. For example, `a b c * +`

Precedence and Associativity

- **Precedence**: When an operand is in between two different operators, which operator will take the operand first, is decided by the precedence of an operator over others. For example, a + b * c → a + (b * c). As multiplication operation has precedence over addition, b * c will be evaluated first.

- **Associativity**: Associativity is the left-to-right or right-to-left order for grouping operands to operators that have the same precedence appear in an expression. For example, in expression a / b * c, both / and * have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators. Here, both / and * are *left-to-right associative*, so the expression will be evaluated as (a / b) * c.

Precedence and Associativity

- Precedence and associativity determines the order of evaluation of an expression.

Operators	*Precedence*	*Associativity*
^, unary +, unary –	6	right-to-left
*, /	5	left-to-right
+, –	4	left-to-right
<, <=, >=, =, !=	3	left-to-right
and	2	left-to-right
or	1	left-to-right

- Parenthesis () can be considered as highest precedence. However, () are not operators in true sense, rather parenthesis states which portion of the expression to be evaluated first.

Algorithm to convert infix to postfix

A simple algorithm for producing postfix from infix:

1. Fully parenthesize the expression;
2. Move all operators so that they replace their corresponding right parentheses;
   - ❖ Replace let parentheses in case infix to prefix
3. Delete all parentheses.

For example: **a / b ^ c + d * e – a * c**

(((a / (b ^ c)) + (d * e)) – (a * c))

a b c ^ / d e * + a c * –

A few examples

Infix	Postfix
A + B	A B +
A + B – C	A B + C –
(A + B) * (C – D)	A B + C D – *
A ^ B * C – D + E / F / (G+H)	A B ^ C * D – E F / G H + / +
((A + B) * C – (D – E)) ^ (F + G)	A B + C * D E – – F G + ^
A – B / (C * D ^ E)	A B C D E $ * / –

Infix	Prefix
A + B	+A B
A + B – C	– + A B C
(A + B) * (C – D)	* + A B – C D
A ^ B * C – D + E / F / (G+H)	+ – * ^ A B C D / / E F + G H
((A + B) * C – (D – E)) ^ (F + G)	^ – * + A B C – D E + F G
A – B / (C * D ^ E)	– A / B * C ^ D E

Evaluating a Postfix Expression

Input: 6 2 3 + – 3 8 2 / + * 2 ^3 +

```
1.  stk = the empty stack;
2.  /* scan the input string reading one element at a
    time into symb */
3.  While (not end of input) {
4.          symb = next input symbol ;
5.          if (symb is an operand)
6.                  push( stk, symb)
7.          else {
8.              /* symb is a operator */
9.              opnd2 = pop(stk);
10.             opnd1 = pop(stk);
11.             value = result of applying symb to
    opnd1 and opnd2 (according to the associativity)
12.             push (stk, value);
13.         } /* end else */
14. } /* end while */
15. return pop(stk);
```

symb	opnd1	opnd2	value	stk
6				6
2				6,2
3				6,2,3
+	2	3	5	6,5
–	6	5	1	1
3				1,3
8				1,3,8
2				1,3,8,2
/	8	2	4	1,3,4
+	3	4	7	1,7
*	7	1	7	7
2				7,2
^	7	2	49	49
3				49,3
+	49	3	52	52

Converting Infix expression to Postfix

```
1.  stk = the empty stack;
2.  /* scan the input string reading one element at a time into symb */
3.  while (not end of input) {
4.          symb = next input symbol ;
5.          if (symb is an operand)
6.                  add symbol to postfix string
7.          else {
8.              while(!empty(stk) && PREC(stackTop(stk)) >= PREC(symb)) {
9.                  add topSymb = pop(stk) in the postfix string;
10.             } /* end while */
11.             push(stk, symb);
12.         } /* end else */
13. } /* end while */
14. /* Output any remaining operators in stack */
15.  while (!empty(stk)) {
16.         topSymb = pop(stk);
17.         add topSymb in the postfix string;
18. } /* end while*/
```

Ex1: Input: a + b * c

symb	stk	prefixStr
a		a
+	+	a
b	+	a b
*	+, *	a b
c	+, *	a b c
		a b c * +

Ex2: Input: a * b + c

symb	stk	prefixStr
a		a
*	*	a
b	*	a b
+	+	a b *
c	+	a b * c
		a b * c +

Taking care of Parenthesis

```
1.  stk = the empty stack;
2.  /* scan the input string reading one element at a time into symb */
3.  while (not end of input) {
4.          symb = next input symbol ;
5.          if (symb is an operand)
6.                  add symbol to postfix string
7.          else if ( symb == ')' ) {
8.              while (!empty(stk) && ( topSymb = pop(stk) != '(' ) ) {
9.                  add topSymb in the postfix string;
10.             } /* end while */
11.         } /* end of else if */
12.         else {
13.             while(!empty(stk) && PREC(stackTop(stk)) >= PREC(symb)) {
14.                 add topSymb = pop(stk) in the postfix string;
15.             } /* end while */
16.             push(stk, symb);
17.       } /* end else */
18. } /* end while */
19. /* Output any remaining operators in stack */
```

Ex3: Input: a * (b + c) * d

symb	stk	prefixStr
a		a
*	*	a
(	*, (	a
b	*, (	a b
+	*, (, +	a b
c	*, (, +	a b c
)	*	a b c +
*	*	a b c + *
d	*	a b c + * d
		a b c + * d *

- In the context of this algorithm, consider the precedence of '(' as small as 0

Problem with right-to-left Associativity

- Consider the expression `a ^ b ^ c`, which is right to left associativity.
- Therefore, it is equivalent to

  `(a ^ (b ^ c))`
- The postfix expression of this expression is `a b c ^ ^`
- But the stated algorithm will result in wrong postfix expression `a b ^ c ^`

symb	stk	prefixStr
a		a
^	^	a
b	^	a b
^	^	a b ^
c	^	a b ^ c
		a b ^ c ^

Taking care of right-to-left associativity

```
1.  stk = the empty stack;
2.  /* scan the input string reading one element at a time into symb */
3.  While (not end of input) {
4.          symb = next input symbol ;
5.          if (symb is an operand)
6.                  add symbol to postfix string
7.          else if ( symb == ')' ) {
8.              while (!empty(stk) && ( topSymb = pop(stk) != '(' ) ) {
9.                  add topSymb in the postfix string;
10.             } /* end while */
11.         } /* end of else if */
12.         else {
13.             while(!empty(stk) && ISP(stackTop(stk)) >= ICP(symb)) {
14.                 add topSymb = pop(stk) in the postfix string;
15.             } /* end while */
16.             push(stk, symb);
17.       } /* end else */
18. } /* end while */
19. /* Output any remaining operators in stack */
```

In-Stack Priority (ISP) and
In-Coming Priority (ICP)

symb	ISP	ICP
)	None	None
^	3	4
*, /	2	2
+,	1	1
(	0	4

Converting Postfix expression to Infix

Do It Yourself